

HyPT: Hybrid Period-Transformer for Long-Term Electricity Forecasting

Gianmarco Fossato[†]

Abstract—Long-term multivariate time series forecasting is central to applications such as electricity load planning, where hundreds of correlated meters must be predicted jointly over long horizons. TimesNet, a strong general-purpose forecasting backbone, models each variate independently through a two-dimensional reshaping of its periodicities, which becomes computationally expensive and ignores cross-variate correlations as the number of channels grows. We propose Hybrid Period-Transformer (HyPT), a patch-based encoder that combines two complementary branches: a *factorized* periodicity branch that compresses N channels into $K \ll N$ learned prototype sequences before applying TimesNet-style Fourier-guided 2D convolution, and a cross-variate attention branch, inspired by iTransformer, that models pairwise dependencies among all channels. The two branches are combined through a learned gate, regularized by a dynamic, per-sample branch-dropout schedule that prevents either branch from dominating, and are complemented by a parallel linear trend path and reversible instance normalization (RevIN). We evaluate HyPT on the Electricity dataset against DLinear, TimesNet, iTransformer, TimeMixer and TimeXer, all reproduced under an identical Time-Series Library (TSLib)-based pipeline, and tune HyPT’s hyperparameters with an Optuna-based search that combines a tree-structured Parzen estimator (TPE) sampler with Hyperband pruning. After tuning, HyPT achieves the best average mean squared error (MSE) and mean absolute error (MAE) among the six models (0.171 / 0.264), is the single best model on MAE at every forecast horizon, and is essentially tied with the strongest baseline, TimeXer, on MSE. Ablation studies, averaged over three random seeds per horizon, further isolate the contribution of each branch and of the dynamic branch-dropout regularizer.

Index Terms—Time Series Forecasting, Multivariate Forecasting, Transformers, Periodicity Modeling, Attention Mechanisms, Electricity Load Forecasting

I. INTRODUCTION

Time series forecasting underlies many real-world decision processes, from weather prediction to industrial maintenance, and among these, electricity consumption forecasting is particularly consequential: accurate long-horizon predictions of hundreds of correlated meters directly affect grid balancing and energy planning. Real-world series such as these mix short-term periodic patterns with slow trends and strong dependencies across channels, so the choice of temporal and cross-variate inductive bias has a large effect on model accuracy.

TimesNet [1] is a strong general-purpose backbone for this setting, built around a per-channel Fourier-guided 2D convolution that captures each variate’s dominant periodicities (detailed in Section II). However, this transform-and-convolve

step runs independently per channel, so its cost scales with the number of variates N ; on datasets with hundreds of correlated channels, such as Electricity’s 321 clients, this is both computationally wasteful and blind to the cross-variate correlations that recent Transformer-based models, such as iTransformer [2], exploit explicitly via attention over per-variate tokens.

In this project we ask whether the strengths of these two families, factorized intra-variate periodicity modeling and explicit cross-variate attention, can be combined without inheriting their respective costs. We propose the *Hybrid Period-Transformer* (HyPT), a patch-based hybrid encoder for long-term forecasting whose core novelty is a *prototype factorization* of TimesNet’s periodicity block: instead of running the Fourier-guided 2D convolution independently on all N channels, HyPT first soft-clusters the channels into $K \ll N$ learned prototype sequences, applies the expensive processing only to these K sequences, and then redistributes the result back to the original channels. This branch runs in parallel with an iTransformer-style cross-variate attention branch, and the two are fused through a learned gate regularized by a dynamic, per-sample branch-dropout schedule that prevents either branch from dominating during training. We evaluate HyPT exclusively on long-term forecasting for the Electricity dataset against DLinear, TimesNet, iTransformer, TimeMixer and TimeXer, all reproduced inside the same Time-Series Library (TSLib)-derived pipeline for a fair comparison, and tune its hyperparameters with an Optuna-based search.

Our contributions are as follows:

- A *prototype-factorized periodicity block* that reduces the per-layer cost of TimesNet’s 2D-variation modeling by operating on $K \ll N$ learned prototype sequences within a lower-dimensional patch space (detailed in Section V), decoupling the temporal periodicity modeling from the total number of channels N .
- A *hybrid two-branch encoder* that combines this factorized periodicity branch with an iTransformer-style cross-variate attention branch through a learned gate.
- A *dynamic per-sample branch-dropout* regularization scheme, with a warm-up schedule, that discourages the model from collapsing onto a single branch.
- An *ablation study* isolating the contribution of each branch and of the dynamic branch-dropout regularizer, averaged over three random seeds per horizon.
- An *Optuna-based hyperparameter search* over 13 architectural and optimization hyperparameters, tuned once and reused unchanged across all forecast horizons.

[†]Student ID: 2185061,
email: gianmarco.fossato@studenti.unipd.it

The rest of this report is organized as follows. Section II reviews the baselines and building blocks HyPT is built upon. Section III describes the end-to-end processing pipeline. Section IV details the Electricity dataset, its preprocessing, and the train/validation/test split. Section V presents the HyPT architecture and the hyperparameter-tuning procedure. Section VI reports the main forecasting results, an ablation isolating the contribution of each branch, the Optuna hyperparameter search, and a computational-cost discussion. Section VII concludes.

II. RELATED WORK

TimesNet [1] discovers, for each channel independently, the dominant periods via the fast Fourier transform (FFT) amplitude spectrum, reshapes the 1D series into 2D tensors (columns: intra-period variation, rows: inter-period variation), and processes them with a parameter-efficient inception block, aggregating per-period representations by amplitude-weighted averaging. TimesNet’s own ablations show this inception block can be replaced by stronger 2D backbones such as ConvNeXt for further gains; HyPT’s periodicity branch adopts this substitution directly.

DLinear [3] showed a single linear layer mapping the look-back window to the forecast horizon (optionally after a trend/seasonal split) is a remarkably strong, cheap baseline, motivating HyPT’s parallel linear trend path rather than relying purely on nonlinear processing for the slow-moving component.

iTransformer [2] inverts standard Transformer tokenization: rather than one token per time step across variates, it embeds each variate’s entire series into a single token and applies self-attention across the N variate tokens, letting the feed-forward network model temporal patterns while attention captures multivariate correlations directly. HyPT’s cross-variate branch mirrors this design at a patch-summary level.

TimeMixer [4] and *TimeXer* [5] are two further, more recent points of comparison. TimeMixer is a fully multilayer perceptron (MLP)-based architecture decomposing series into seasonal and trend components at multiple scales, mixed bottom-up and top-down, avoiding attention altogether while remaining competitive. TimeXer augments a Transformer with a patch-level self-attention stream for the target series and a variate-level cross-attention stream, bridged by learned global tokens, originally to inject exogenous information but also effective as a strong general multivariate forecaster.

HyPT additionally builds on three components that are not forecasting models in themselves: *reversible instance normalization* (RevIN) [6], which removes and later restores each window’s mean/variance to counteract distribution shift; patch-based tokenization as in *PatchTST* [7], segmenting each channel-independent series into overlapping patches before embedding; and *ConvNeXt* [8], a modernized convolutional block with a large-kernel depthwise convolution and inverted bottleneck, used here in place of TimesNet’s Inception block.

Taken together, these works define two largely separate paradigms: convolutional, Fourier-guided periodicity model-

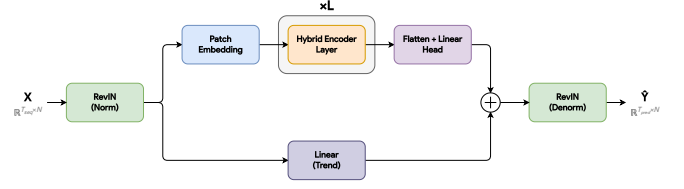


Fig. 1: End-to-end HyPT processing pipeline. The hybrid encoder stack is detailed in Fig. 2.

ing (TimesNet) and explicit cross-variate attention (iTransformer), with TimeMixer and TimeXer as further points of comparison. Rather than replacing either paradigm, HyPT makes the first affordable at the scale of Electricity’s 321 channels through prototype factorization, and combines it with the second in a single hybrid encoder built on the shared RevIN, PatchTST-style patching, and ConvNeXt building blocks reviewed above.

III. PROCESSING PIPELINE

Fig. 1 shows the end-to-end flow: shell scripts set all architectural and training hyperparameters and pass them to `run.py`, which dispatches to the `Exp_Long_Term_Forecast` experiment class, building the Electricity data loaders, instantiating the selected model (HyPT or a baseline), and driving the train/validation/test loop. At the model level, HyPT first removes each window’s instance-level statistics with RevIN, tokenizes the normalized series into overlapping per-channel patches, and processes them through a stack of Hybrid Encoder Layers (Section V, Fig. 2), flattened and projected to the forecast horizon by a linear head; in parallel, a per-channel linear layer maps the normalized look-back window directly to the horizon as a trend estimate, added to the encoder’s output before RevIN denormalization.

IV. SIGNALS AND FEATURES

A. Measurement Setup and Raw Data Format

This work evaluates forecasting on the Electricity benchmark, sourced from the University of California, Irvine (UCI) Machine Learning Repository (*ElectricityLoadDiagrams20112014*) and distributed pre-cleaned within the TSLib [9]. It records hourly consumption (kW) across $N = 321$ clients from 2012–2014, on a constant 1-hour grid with no missing timestamps or NaNs (already gap-filled and re-interpolated by TSLib). The raw matrix $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{T_{\text{total}} \times (N+1)}$ holds a timestamp column plus 321 series of length $T_{\text{total}} = 26\,304$ hours; all 321 variables are evaluated jointly under a multivariate-to-multivariate (M) setting, serving as both inputs and targets.

B. Pre-processing

Because the TSLib release of Electricity is already gap-free and on a regular hourly grid, no re-interpolation, gap-filling, or artifact-removal is applied; the only pre-windowing steps are (i) reordering columns so the target variable OT is last, and

(ii) excluding the raw timestamp column, encoded separately into the calendar-feature tensor below. No outlier clipping or smoothing is applied, keeping the comparison consistent with prior TSLib-based baselines.

C. Standardization and Time-Feature Extraction

To prevent data leakage across evaluation splits, channel-wise zero-mean and unit-variance standardization is computed exclusively on the training split prior to sliding-window extraction. Each channel $n \in \{1, \dots, N\}$ is scaled according to

$$\mathbf{X}_{\text{scaled}} = \frac{\mathbf{X}_{\text{full}} - \boldsymbol{\mu}_{\text{train}}}{\boldsymbol{\sigma}_{\text{train}}}, \quad (1)$$

where $\mathbf{X}_{\text{full}} \in \mathbb{R}^{T_{\text{total}} \times N}$ is the raw numerical feature matrix (i.e., $\mathbf{X}_{\text{raw}}$ with the date column removed), and $\boldsymbol{\mu}_{\text{train}} \in \mathbb{R}^{1 \times N}$ and $\boldsymbol{\sigma}_{\text{train}} \in \mathbb{R}^{1 \times N}$ are the channel-wise mean and standard deviation vectors computed over the training split, broadcast along the time dimension.

Each look-back window of length T_{seq} is also paired with a temporal feature matrix $\mathbf{X}_{\text{mark}} \in \mathbb{R}^{T_{\text{seq}} \times 4}$ from TSLib's `time_features` utility (`embed=timeF`), encoding normalized month, day-of-month, day-of-week, and hour indicators; it is emitted globally for pipeline uniformity, though architectures integrate it differently based on their tokenization.

D. Windowing and Per-Window Feature Vectors

Samples are extracted by sliding a window of stride 1 over the normalized sequence. For each sample index i , an input look-back tensor $\mathbf{X} \in \mathbb{R}^{T_{\text{seq}} \times N}$ and a target tensor $\mathbf{Y} \in \mathbb{R}^{(T_{\text{label}}+T_{\text{pred}}) \times N}$ are constructed as

$$\mathbf{X} = \mathbf{X}_{\text{scaled}}[i : i + T_{\text{seq}}], \quad (2)$$

$$\mathbf{Y} = \mathbf{X}_{\text{scaled}}[i + T_{\text{seq}} - T_{\text{label}} : i + T_{\text{seq}} + T_{\text{pred}}], \quad (3)$$

with $T_{\text{seq}} = 96$ hours (4 days), $T_{\text{label}} = 48$ hours, and $T_{\text{pred}} \in \{96, 192, 336, 720\}$ hours (4, 8, 14 and 30 days respectively). Synchronous time-mark tensors $\mathbf{X}_{\text{mark}} \in \mathbb{R}^{T_{\text{seq}} \times 4}$ and $\mathbf{Y}_{\text{mark}} \in \mathbb{R}^{(T_{\text{label}}+T_{\text{pred}}) \times 4}$ are extracted over the same window boundaries. The per-window feature vector fed to HyPT (and to all baselines, for fairness) is therefore the tuple $(\mathbf{X}, \mathbf{X}_{\text{mark}})$, with $\mathbf{X}$ being the only tensor consumed by HyPT's encoder (HyPT is encoder-only and ignores $\mathbf{Y}_{\text{mark}}$ and the decoder-side `label` prefix, kept only for application programming interface (API) compatibility with TSLib).

E. Dataset Splitting

Following the standard TSLib protocol for Electricity [10], the sequence is split chronologically (no shuffling, to preserve temporal integrity) into training (70%), validation (10%), and test (20%) ranges; windows are drawn in randomized mini-batch order within splits during training/validation, while test windows are evaluated sequentially. The split is identical for all compared models, so every reported metric is computed on the same held-out test windows.

V. LEARNING FRAMEWORK

A. Overall model

Given a look-back window $\mathbf{X} \in \mathbb{R}^{T_{\text{seq}} \times N}$ ($T_{\text{seq}} = \text{seq_len}$, $N = 321$; omitting the batch dimension for brevity), HyPT computes

$$\mathbf{X}' = \text{RevIN}_{\text{norm}}(\mathbf{X}), \quad (4)$$

$$\mathbf{Y}_{\text{trend}} = \text{Linear}_{T_{\text{seq}} \rightarrow T_{\text{pred}}}(\mathbf{X}'^{\top}) \in \mathbb{R}^{N \times T_{\text{pred}}}, \quad (5)$$

$$\mathbf{Z}^0 = \text{PatchEmbed}(\mathbf{X}'^{\top}) \in \mathbb{R}^{N \times P \times D}, \quad (6)$$

$$\mathbf{Z}^{\ell} = \text{HybridEncoderLayer}(\mathbf{Z}^{\ell-1}), \ell = 1, \dots, L, \quad (7)$$

$$\mathbf{Y}_{\text{pred}} = \text{Linear}_{P \cdot D \rightarrow T_{\text{pred}}}(\text{Dropout}(\text{flatten}(\mathbf{Z}^L))) + \mathbf{Y}_{\text{trend}} \in \mathbb{R}^{N \times T_{\text{pred}}}, \quad (8)$$

$$\hat{\mathbf{Y}} = \text{RevIN}_{\text{denorm}}(\mathbf{Y}_{\text{pred}}^{\top}), \quad (9)$$

where $T_{\text{pred}} = \text{pred_len}$, P is the number of patches, $D = \text{d_model}$, $L = \text{e_layers}$. Here, $\mathbf{Y}_{\text{pred}}^{\top}$ denotes the transpose of $\mathbf{Y}_{\text{pred}}$, swapping the node and horizon axes to match the output shape. Patches are extracted per channel with length `patch_len` and stride `patch_len/2`, following PatchTST-style channel-independent tokenization [7].

B. Hybrid Encoder Layer

Each Hybrid Encoder Layer (Fig. 2) receives $\mathbf{Z} \in \mathbb{R}^{N \times P \times D}$ and computes two branch outputs $\mathbf{H}_A, \mathbf{H}_B \in \mathbb{R}^{N \times P \times D}$ in parallel.

Branch A - Prototype Period Factorization. The core idea is to avoid running TimesNet's expensive Fourier-guided 2D convolution independently on all N channels by instead running it on $K = 16$ learned prototype sequences, with $K \ll N$. Concretely,

$$\tilde{\mathbf{Z}}_n = \mathbf{Z}_n \mathbf{W}_{\text{down}}^{\top} \in \mathbb{R}^{P \times d_p}, \quad (10)$$

$$\mathbf{c}_n^{\text{temp}} = \frac{1}{P} \sum_{p=1}^P \tilde{\mathbf{Z}}_{n,p}, \quad (11)$$

$$\mathbf{c}_n^{\text{spec}} = \frac{1}{P_{\text{fft}}} \sum_{f=0}^{P_{\text{fft}}-1} |[\text{rFFT}(\tilde{\mathbf{Z}}_n)]_f|, \quad (12)$$

$$\mathbf{q}_n = [\mathbf{c}_n^{\text{temp}} \parallel \mathbf{c}_n^{\text{spec}}] \mathbf{W}_q^{\top}, \quad (13)$$

$$\pi_{n,j} = \text{softmax}_n \left(\frac{\mathbf{q}_n \cdot \mathbf{e}_j}{\sqrt{d_p}} \right), \quad (14)$$

$$\rho_{n,j} = \text{softmax}_j \left(\frac{\mathbf{q}_n \cdot \mathbf{e}_j}{\sqrt{d_p}} \right), \quad (15)$$

$$\mathbf{R}_j = \sum_{n=1}^N \pi_{n,j} \tilde{\mathbf{Z}}_n \in \mathbb{R}^{P \times d_p}, \quad (16)$$

where $P_{\text{fft}} = \lfloor P/2 \rfloor + 1$, $d_p = d_{\text{period}}$, $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_p \times D}$, $\mathbf{W}_q \in \mathbb{R}^{d_p \times 2d_p}$, $\mathbf{W}_{\text{up}} \in \mathbb{R}^{D \times d_p}$, and $\mathbf{e}_j \in \mathbb{R}^{d_p}$ are K learnable prototype keys; $\mathbf{c}_n^{\text{temp}}$ and $\mathbf{c}_n^{\text{spec}}$ are the per-channel temporal-mean and FFT-amplitude profiles concatenated into the channel summary, $\pi_{n,j}$ pools the N channels into K prototypes, and $\rho_{n,j}$ redistributes prototype outputs back to channels. Each prototype sequence $\mathbf{R}_j$ is processed following TimesNet's FFT-and-reshape procedure, but with periods selected independently per prototype: for each prototype, the top- k periods are found from its batch-averaged FFT

amplitude spectrum, $\mathbf{R}_j$ is reshaped into k 2D grids using this period set, each grid passes through the ConvNeXt block [8] for its period rank (shared across all K prototypes, so parameter count is independent of K), and results are recombined via a softmax over the per-sample amplitudes at the selected periods – period *selection* is thus batch-level while combination *weights* are per-sample. The K processed prototypes $\{\mathbf{O}_j\}$ are recombined and projected back,

$$(\mathbf{H}_A)_n = \left(\sum_{j=1}^K \rho_{n,j} \mathbf{O}_j \right) \mathbf{W}_{\text{up}}^\top \in \mathbb{R}^{P \times D} \quad \text{for } n = 1, \dots, N \quad (17)$$

Branch B - Cross-Variate Attention. Patches are mean-pooled into one summary token per channel, $\mathbf{s}_n = \left(\frac{1}{P} \sum_{p=1}^P \mathbf{z}_{n,p} \right) \mathbf{W}_{\text{tok}}^\top$ ($\mathbf{W}_{\text{tok}} \in \mathbb{R}^{D \times D}$ a learned projection), and a standard multi-head self-attention layer [11] is applied across the N tokens, following iTransformer’s inverted-attention idea [2]. Attended tokens pass through a near-zero-initialized output projection $\mathbf{W}_{\text{proj}}$ and are broadcast back over the P patches to form $\mathbf{H}_B$.

Gated fusion with dynamic branch dropout. A learned gate $\mathbf{g} = \sigma(\gamma) \in \mathbb{R}^D$ combines the branches,

$$\mathbf{H}_{\text{fused}} = \mathbf{g} \odot \mathbf{H}_A + (1 - \mathbf{g}) \odot \mathbf{H}_B. \quad (18)$$

During training, this fused output is regularized with a per-sample *branch dropout*: with probability $p_t/2$ each, a sample uses $\mathbf{H}_A$ or $\mathbf{H}_B$ alone instead of the gated combination, where p_t is linearly warmed up from 0 to a target rate `branch_dropout` over the first `branch_warmup_epochs` epochs, discouraging early collapse onto a single branch while still letting the gate specialize later. The resulting $\mathbf{H}_{\text{drop}}$ is added to the residual stream, layer-normalized, passed through a two-layer Gaussian error linear unit (GELU) feed-forward neural network (FFNN), and layer-normalized again, following a standard post-norm Transformer block:

$$\mathbf{Z}' = \text{Norm}_1(\mathbf{Z} + \text{Dropout}(\mathbf{H}_{\text{drop}})), \quad (19)$$

$$\mathbf{Z}_{\text{out}} = \text{Norm}_2(\mathbf{Z}' + \text{Dropout}(\text{FFNN}(\mathbf{Z}'))). \quad (20)$$

C. Ablation modes

The two branches can be individually disabled through the `ablation_mode` flag, allowing `branch_a` (periodicity only), `branch_b` (cross-variate only), `both` (full hybrid), and `none` (branches bypassed) variants to be trained. This modular design is the basis for the ablation study in Section VI-C.

D. Computational Complexity

Branch A factors the expensive 2D variation modeling from all N channels onto $K \ll N$ prototypes. Operating in a bottleneck dimension $d_p \ll D$ and on patches of length $P \ll T_{\text{seq}}$, the per-layer cost reduces from TimesNet’s $\mathcal{O}(N \cdot D \cdot T_{\text{seq}} \log T_{\text{seq}} + N \cdot k \cdot T_{\text{seq}} \cdot D^2)$ to $\mathcal{O}(N \cdot P \cdot D \cdot d_p +$

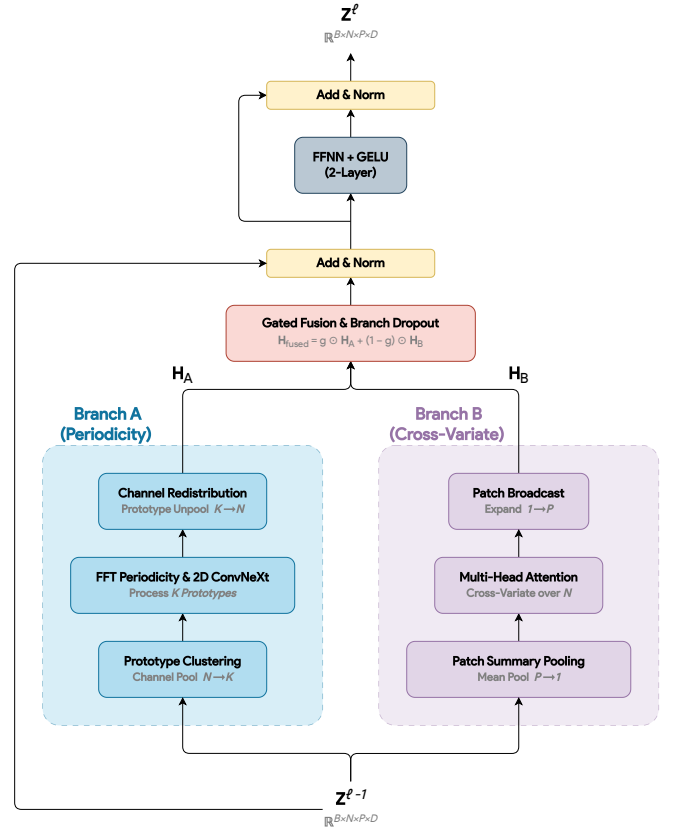


Fig. 2: Internal structure of one Hybrid Encoder Layer.

$N \cdot d_p \cdot P \log P + N \cdot K \cdot P \cdot d_p + K \cdot d_p \cdot P \log P + K \cdot k \cdot P \cdot d_p^2$), decoupling periodicity modeling from N and T_{seq} . Branch B adds a comparatively cheap $\mathcal{O}(N^2 \cdot D + N \cdot D^2 + N \cdot P \cdot D)$ term. Wall-clock comparisons are in Section VI-E.

E. Optimization and hyperparameter tuning

All models are trained with Adam [12] minimizing mean squared error (MSE) loss, with early stopping on validation loss. HyPT’s hyperparameters are tuned with Optuna [13] (default tree-structured Parzen estimator (TPE) sampler plus a HyperbandPruner [14] that prunes unpromising trials early from the per-epoch validation loss), over a 13-hyperparameter search space (learning rate (LR), batch size, epochs, `d_model`, `d_ff`, attention heads, `patch_len`, `dropout`, `top_k`, `branch_dropout`, warm-up length, encoder layers, `d_period`). Rather than searching per horizon, the search runs once on `pred_len=192` as a practical middle-ground, and the resulting configuration is reused unchanged for all four horizons in the main comparison (Section VI-D, Tab. 1).

VI. RESULTS

A. Experimental setup

All six models are trained and evaluated inside the same pipeline under the same 70%/10%/20% chronological split described in Section IV, so performance differences can be

TABLE 1: Hyperparameters of all compared models on Electricity (seq_len=96). Baselines follow the original TSLib shell scripts; HyPT is tuned via Optuna on horizon 192 (Section VI-D).

Hyperparameter	HyPT (ours)	TimesNet	iTransformer	TimeXer	TimeMixer	DLinear
d_model	256	256	512	512	16	—
d_ff	512	512	512	512-2048 ^a	32	—
e_layers	2	2	3	3-4 ^b	3	2
d_layers	1	1	1	1	1	1
d_period	32	—	—	—	—	—
top_k	3	5 ^c	—	—	—	—
patch_len	16	—	—	16	—	—
Batch size	16	32	16	4	32	32
LR	10^{-3}	10^{-4}	5×10^{-4}	10^{-4}	10^{-2}	10^{-4}
LR decay	type3	type1	type1	type1	type1	type1
branch_dropout	0.15	—	—	—	—	—
branch_warmup_epochs	2	—	—	—	—	—

^a TimeXer uses d_ff = 512 at horizon 96, and d_ff = 2048 at horizons 192, 336, and 720.

^b TimeXer uses e_layers = 4 at horizons 96 and 336, e_layers = 3 at 192 and 720.

^c TimesNet reduces top_k to 3 at horizon 720.

attributed to the models rather than the evaluation protocol: Electricity ($N = 321$, look-back 96), multivariate-to-multivariate (M) setting, horizons {96, 192, 336, 720}, evaluated via MSE and mean absolute error (MAE).

The execution configurations of the baselines, as defined in their shell scripts, are summarized in Tab. 1.

It is worth stressing an asymmetry in how models were tuned: the five baselines use the hyperparameters from their original TSLib scripts (Tab. 1), chosen to generalize across benchmarks rather than to be optimal on Electricity, whereas HyPT was tuned specifically on Electricity via Optuna (Section VI-D). Early-stopping patience is 3 for the Optuna-tuned HyPT used in the main comparison and 5 for the hand-chosen configuration in the ablation study (Section VI-C); baselines use their scripts’ patience (typically 3). HyPT uses LR 10^{-3} with a delayed type3 decay ($\eta_t = \eta_0$ for $t < 3$, $\eta_t = \eta_0 \cdot 0.9^{t-3}$ thereafter), preserving a higher LR during initial joint branch adaptation, while baselines use the default type1 exponential decay and their own scripts’ LRs. Therefore, Tab. 2 should be read as “best dataset-specific configuration vs. best general-purpose configuration,” not as a fully hyperparameter-matched comparison.

B. Main forecasting results

Tab. 2 compares HyPT against the five baselines across all four forecast horizons. HyPT obtains the best average MSE and MAE overall (0.171/0.264), matching TimeXer’s rounded average MSE while leading on average MAE (0.264 vs. 0.269), and with a clearer margin over iTransformer, TimeMixer, TimesNet and DLinear on both metrics. HyPT is the single best model on MAE at every horizon; on MSE it alternates the top spot with TimeXer (best at 96 and 336, marginally behind at 192 and 720, always within 0.006), so the two are best understood as essentially tied and jointly ahead of the rest. TimesNet and DLinear remain distinctly behind the top three at every horizon, on both metrics.

Given the tuning asymmetry above, the most informative comparison is against iTransformer and TimeXer, the strongest Transformer-based baselines; TimeXer’s competitiveness despite general-purpose defaults suggests HyPT’s periodicity branch is highly effective on its own, with the attention branch

TABLE 2: Long-term multivariate forecasting performance on Electricity dataset ($N = 321$, seq_len = 96). **Red bold** indicates the best result, **blue underline** indicates the second best.

	HyPT (ours)		TimesNet		iTransformer		TimeXer		TimeMixer		DLinear	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
96	0.137	0.233	0.168	0.272	0.149	<u>0.240</u>	<u>0.140</u>	0.242	0.156	0.247	0.210	0.302
192	<u>0.157</u>	0.251	0.187	0.289	0.164	<u>0.255</u>	0.157	0.256	0.170	0.261	0.210	0.305
336	0.172	0.268	0.203	0.305	0.178	<u>0.271</u>	<u>0.175</u>	0.273	0.188	0.279	0.223	0.319
720	<u>0.216</u>	0.302	0.238	0.331	0.229	0.314	0.210	<u>0.307</u>	0.228	0.313	0.258	0.350
Avg	0.171	0.264	0.199	0.299	0.180	0.270	<u>0.171</u>	<u>0.269</u>	0.185	0.275	0.225	0.319

TABLE 3: Ablation of the two HyPT branches on Electricity, averaged over 3 seeds (2021/2022/2023) per horizon. **Red bold** indicates the best result, **blue underline** indicates the second best.

	None		Branch A		Branch B		Both		Both (no drop)	
Horizon	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
96	0.166	0.255	0.145	0.240	0.141	0.237	0.141	0.237	<u>0.141</u>	<u>0.237</u>
192	0.175	0.262	0.159	0.251	<u>0.160</u>	<u>0.253</u>	0.161	0.254	0.160	0.253
336	0.190	0.278	0.176	0.270	0.177	0.270	0.173	0.267	<u>0.174</u>	<u>0.269</u>
720	0.230	0.312	0.203	<u>0.295</u>	0.212	0.301	<u>0.204</u>	0.294	0.217	0.304
Grand Avg	0.190	0.277	<u>0.171</u>	<u>0.264</u>	0.172	0.265	0.170	0.263	0.173	0.266

adding a small but consistent gain rather than the margin being purely a tuning artifact.

C. Ablation study

To validate the two-branch design independently of the Optuna search, the ablation study uses an initial, hand-chosen configuration (d_period=16, top_k=5, branch_dropout=0.1, patience=5, batch size 32, LR 5×10^{-4}) rather than the Optuna-tuned configuration of Section VI-D, isolating the architectural contributions from dataset-specific tuning. Tab. 3 isolates the contribution of each branch by comparing five variants, obtained through the ablation_mode flag (Section V):

- none: both branches bypassed, fused representation set to zero, model reduces to the patch backbone + linear trend path (floor baseline);
- branch_a: prototype-factorized periodicity branch only;
- branch_b: cross-variate attention branch only;
- both: full gated model with dynamic branch-dropout (p_t warmed up to 0.1 over 3 epochs);
- both (no drop): same gated model with branch dropout disabled ($p_t \equiv 0$), to isolate the contribution of the regularization scheme.

Each variant is trained and evaluated with three random seeds (2021, 2022, 2023) at every horizon, and Tab. 3 reports the resulting per-horizon and grand averages.

Three observations follow. First, both branches individually improve substantially over the no-branch floor at every horizon (e.g. at horizon 96, MSE drops from 0.166 to 0.145 and 0.141 for Branch A and B alone), confirming the gain is not simply coming from the patch backbone or trend path. Second, the two branches are fairly close in isolation, with Branch A slightly ahead at longer horizons (336, 720) and Branch B marginally ahead at 96. Third, the full gated model

achieves the best grand-average MSE/MAE (0.170/0.263), only marginally better than Branch A alone (0.171/0.264); combining the branches without regularization instead degrades long-horizon performance (MSE 0.217 vs. 0.204 at horizon 720), and the dynamic branch-dropout schedule is what recovers this stability.

Across all runs, the learned gate $g = \sigma(\gamma)$ consistently converges near 0.5, possibly for two reasons: it is a single per-feature-channel parameter shared across all $N = 321$ variates, initialized at zero, with only $L = 2$ layers and near-parity between branches giving limited gradient pressure to move away from initialization; and branch-dropout explicitly forces each branch to remain independently competent, preventing collapse toward either extreme.

D. Hyperparameter search

We conducted a 38-trial Optuna search on the `pred_len=192` horizon (21 completed, 15 pruned by Hyperband, 2 interrupted) using the TPE sampler, reaching a best validation MSE of 0.1294. Relative to the ablation baseline (Tab. 1), the search favored a doubled prototype dimension (`d_period=32` vs. 16), fewer periods (`top_k=3` vs. 5), a higher LR (10^{-3} vs. 5×10^{-4}), a smaller batch size (16 vs. 32), and a shorter, stronger branch-dropout schedule (0.15 over 2 warm-up epochs vs. 0.1 over 3).

Hyperparameter importance analysis identifies warm-up length (`branch_warmup_epochs`, importance ≈ 0.49) and LR (≈ 0.19) as the dominant drivers, with architectural dimensions (`d_model`, `d_ff`, `top_k`, `patch_len`) contributing little. Consistent with this, 2-epoch warm-up runs completed more often (61.5% vs. 33.3%) and achieved lower mean validation MSE (0.133 vs. 0.154) than 5-epoch runs, matching Section VI-C’s finding that prolonged warm-ups leave too few epochs for branch-dropout to reach its target rate before early stopping.

Pruner and sampler behavior reinforce this: Hyperband pruned all 5 batch-size-64 trials (slow early-epoch convergence), while completed trials concentrated at the 10^{-3} LR ceiling (14 of 21) with the lowest MSE there, suggesting the upper bound could be pushed further in future search.

E. Computational cost

Beyond accuracy, HyPT targets computational efficiency (complexity analyzed in Section V): Branch A factors the expensive FFT-guided 2D convolution from $N = 321$ channels onto $K = 16$ prototypes, decoupling cost from the forecast horizon `pred_len`, unlike TimesNet which couples cost to `seq_len`; Branch B adds only a comparatively cheap $\mathcal{O}(N^2 \cdot D + N \cdot D^2 + N \cdot P \cdot D)$ cross-variate attention term.

Empirically, despite a smaller batch size and no `torch.compile` (unlike TimesNet), HyPT achieves a 2.7–3.9 $\times$ per-epoch speedup at longer horizons (192–720), an advantage that widens with horizon length, validating the `pred_len`-decoupling design. For reference, DLinear remains fastest overall (under 1s/epoch), iTransformer and TimeMixer fall in between, and TimeXer’s per-epoch cost is comparable to HyPT.

VII. CONCLUDING REMARKS

This project introduced HyPT, a hybrid encoder for long-term Electricity forecasting that fuses a prototype-factorized periodicity branch (Branch A) with a cross-variate attention branch (Branch B) through a gated, branch-dropout-regularized combination. Tuned via Optuna, HyPT achieves the best average MSE/MAE among the six compared models, tying with TimeXer on MSE and leading on MAE at every horizon; this should be read together with the tuning asymmetry noted in Section VI, since HyPT was tuned specifically on Electricity while several baselines used general-purpose defaults.

The main lesson from this project is that, on Electricity, intra-variate periodicity and cross-variate correlation are largely overlapping information sources: both branches are independently strong and nearly redundant in isolation, so combining them yields only a marginal further gain and requires branch-dropout regularization to avoid interference at long horizons. The main practical difficulty we encountered was computational rather than architectural: Branch A’s prototype-assignment step currently relies on an unvectorized Python loop, which, despite HyPT’s favorable per-epoch speedup over TimesNet, constrained our Optuna search to 38 trials on a single horizon. Vectorizing this step and evaluating HyPT against hyperparameter-matched baselines on further datasets are natural next steps.

REFERENCES

- [1] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, “Timesnet: Temporal 2d-variation modeling for general time series analysis,” Apr. 2023.
- [2] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, “itransformer: Inverted transformers are effective for time series forecasting,” Mar. 2024.
- [3] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?,” Aug. 2022.
- [4] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J. Y. Zhang, and J. Zhou, “Timemixer: Decomposable multiscale mixing for time series forecasting,” May 2024.
- [5] Y. Wang, H. Wu, J. Dong, G. Qin, H. Zhang, Y. Liu, Y. Qiu, J. Wang, and M. Long, “Timexer: Empowering transformers for time series forecasting with exogenous variables,” Mar. 2024.
- [6] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, “Reversible instance normalization for accurate time-series forecasting against distribution shift,” in *International Conference on Learning Representations*, Feb. 2023.
- [7] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” Mar. 2023.
- [8] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A convnet for the 2020s,” Mar. 2022.
- [9] A. Trindade, “ElectricityLoadDiagrams20112014,” UCI Machine Learning Repository, Dec. 2015.
- [10] Y. Wang, H. Wu, J. Dong, Y. Liu, M. Long, and J. Wang, “Deep time series models: A comprehensive survey and benchmark,” July 2024.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” Aug. 2023.
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” Jan. 2017.
- [13] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” July 2019.
- [14] K. Jamieson and A. Talwalkar, “Non-stochastic best arm identification and hyperparameter optimization,” Feb. 2015.